

INTERNAL ENGINEERING SPECIFICATION

Placement AI (PAI) Document Module Architecture

Complete technical guide: Directory breakdown, data models, worker pipeline, and Vault reconciliation

1. Core Architectural Philosophy

The Fundamental Constitution:

- **AI Never Writes Truth Directly:** Models and OCR engines are only permitted to propose interpretations into a quarantined staging table (`document_candidates` with `review_status = 'pending'`).
- **Human-in-the-Loop Gate:** Extracted facts are only promoted to the canonical **Student Vault** (`vault_values`) when the student confirms them via `POST /api/v1/documents/{id}/review` .
- **Evidence-Backed Profile:** Every fact in the Vault links to immutable source records in `vault_evidence` (document ID, snippet, and confidence).
- **Strict Tenant Isolation:** Storage paths, database queries, and background advisory locks strictly scope to the student's authenticated `person_id` .

2. End-to-End Processing Flow

```
[ 1. CLIENT UPLOAD ]
| POST /api/v1/documents (file: UploadFile, optional documentType)
▼
[ 2. INGESTION & SECURE STORAGE ]
| Validated by: src/pai/intelligences/documents/security/validation.py
| Saved to: Supabase Storage at documents/{person_id}/{doc_id}/{version_id}/original
| Created DB records: `documents` (uploaded) & `document_jobs` (pending)
| Response: 202 Accepted (Non-blocking)
▼
[ 3. ASYNC WORKER POLLING & LEASING ]
| Worker: src/pai/intelligences/documents/workers/analysis_worker.py
| Lock: SELECT c.id FROM document_jobs ... FOR UPDATE SKIP LOCKED
| Advisory Lock: pg_try_advisory_xact_lock(lock_ns, hashtext(document_id))
▼
[ 4. DIGITIZATION & OCR STAGE ]
| Orchestrator: src/pai/intelligences/documents/digitization/service.py
| Provider 1: NativeDocumentProvider (pypdf text extraction from digital PDF/DOCX)
| Provider 2 (Fallback): OpenAIVisionProvider (GPT-4o-mini multimodal OCR for scans/photos)
▼
[ 5. CLASSIFICATION STAGE ]
| Classifier: src/pai/intelligences/documents/classification/classifier.py
| Categorizes: academic, identity, financial, legal (e.g. transcript, passport, diploma)
▼
[ 6. STRUCTURED FACT EXTRACTION ]
| Extractor: src/pai/intelligences/documents/extraction/extractor.py
| Schemas: src/pai/intelligences/documents/extraction/schemas/*.py
| Extracts: GPA, degrees, test bands, institutions, completion dates
| Sensitive fields encrypted using Fernet (VAULT_ENCRYPTION_KEY)
▼
[ 7. IDENTITY MATCHING & RECONCILIATION ]
| Matcher: src/pai/intelligences/documents/identity/matcher.py (compares subject vs student account)
| Engine: src/pai/intelligences/documents/reconciliation/engine.py (reconciles with current Vault)
| Proposals written to: `document_candidates` table (status: pending)
| Conflicts flagged in: `verification_cases` table
▼
[ 8. STUDENT REVIEW & VAULT WRITE GATE ]
| API: POST /api/v1/documents/{document_id}/review
| Write Gate: src/pai/kernel/evidence/vault_apply.py
| Results: Accepted candidates committed to `vault_values` + `vault_evidence`
| Document status updated to 'processed'
```

3. Detailed Directory & Code Map

Every file responsible for document functionality is organized into three architectural layers:

Layer 1: Interfaces & Workers (Network Entrypoints)

File Path	Component Responsibilities & Handlers
src/pai/interfaces/api/documents.py	Exposes the FastAPI document endpoints. Validates onboarding state, handles multipart uploads, streams signed URLs, and processes candidate accept/reject reviews.
src/pai/interfaces/workers/documents.py	CLI process endpoint (<code>py run.py --service documents</code>). Runs the asynchronous loop polling for pending jobs.

Layer 2: Domain Layer (Models, Storage & Business Logic)

File Path	Component Responsibilities & Handlers
src/pai/domains/documents/models.py	Defines SQLAlchemy database entities: <code>Document</code> : Main metadata record (lifecycle, category, status). <code>DocumentVersion</code> : Revision tracking, SHA256 checksums, extracted text. <code>DocumentJob</code> : Worker queue tracking lease, stage, attempts, errors. <code>DocumentCandidate</code> : The Staging Area holding pending AI proposals. <code>DocumentAnalysisRun</code> : Run-level audit record. <code>MessageDocument</code> : Attachment mapping to chat messages.
src/pai/domains/documents/service.py	Contains domain operations: <code>review_document_candidates()</code> , <code>list_documents()</code> , <code>get_document_owned()</code> , and <code>mark_document_deleted()</code> .
src/pai/domains/documents/text.py	Low-level text extraction from binary buffers using <code>pypdf</code> .
src/pai/platform/storage/supabase.py	Manages raw object storage in Supabase private buckets (<code>upload_private</code> , <code>download_bytes</code> , <code>create_signed_download_url</code> , <code>delete_object</code>).
src/pai/kernel/evidence/vault_apply.py	The deterministic gate that writes approved candidates to <code>vault_values</code> , links evidence to <code>vault_evidence</code> , and logs to <code>vault_history</code> .

Layer 3: Intelligences Layer (The Processing Pipeline)

Folder / File	Functionality
intelligences/documents/ingest.py	Immediate upload orchestrator: validates bytes, uploads to Supabase, creates DB entities.
intelligences/documents/pipeline.py	The master analysis pipeline: coordinates digitization, classification, extraction, identity checks, and candidate generation.
digitization/service.py	Routes files between native text parsers and AI vision OCR.
providers/native.py	Zero-latency extraction for digital PDFs, DOCX, and text.
providers/openai_vision.py	Multimodal OCR engine using <code>gpt-4o-mini</code> for scans, photos, and handwritten documents.
classification/classifier.py	Classifies document category and specific type (e.g. transcript, test_score, passport).
extraction/extractor.py	Prompts LLM with structured Pydantic schemas to output strictly typed JSON facts.
extraction/schemas/	Structured Pydantic definitions: <code>transcript.py</code> , <code>degree.py</code> , <code>passport.py</code> , <code>test_score.py</code> , <code>resume.py</code> .
identity/matcher.py	Validates that the document subject name/DOB matches the student account.
normalization/	Normalizes variations into global standards: <code>gpa.py</code> , <code>dates.py</code> , <code>institutions.py</code> , <code>names.py</code> .
reconciliation/engine.py	Compares extracted candidates with existing profile data; creates proposals and flags conflicts.
verification/service.py	Opens, resolves, and tracks <code>verification_cases</code> .
workers/analysis_worker.py	Background daemon polling loop, lease management, and advisory locks.

4. Database Schema Reference

Table Name	Key Columns	Purpose & State
documents	id, person_id, document_type, category, status, storage_path	Master record. Status moves: uploaded → digitizing → classifying → extracting → reconciling → ready / awaiting_review → processed.
document_versions	id, document_id, version_number, sha256, content_text	Tracks revisions, file hash, digitized raw text, and encrypted JSON extraction.
document_jobs	id, document_id, status, attempts, current_stage, locked_at	Worker queue. Status: pending → processing → completed (or failed after 3 attempts).
document_candidates	id, document_id, field_key, value, confidence, review_status	The Staging Area: AI-extracted proposals awaiting human approval (pending , accepted , or rejected).
vault_values	id, vault_id, field_key, value, status, verification_level	The Official Student Vault: Active student profile truth (status = 'active').
vault_evidence	id, vault_value_id, source_reference, evidence_text, confidence	Auditable link connecting a Vault fact back to the document ID and exact text snippet.
verification_cases	id, person_id, document_id, case_type, severity, status	Logged when identity mismatches or critical grade discrepancies are found.

5. Swagger API Quick Reference

Method	Endpoint	Summary
POST	/api/v1/documents	Upload document (returns 202 Accepted immediately).
GET	/api/v1/documents	List all documents for the authenticated student.
GET	/api/v1/documents/{id}/status	Poll worker processing stage and job status.
GET	/api/v1/documents/{id}/candidates	Inspect AI-extracted proposals waiting in the staging area.
POST	/api/v1/documents/{id}/review	Accept or reject candidate proposals into the Student Vault.
GET	/api/v1/documents/{id}	Get metadata, decrypted extraction, and signed download URL.
DELETE	/api/v1/documents/{id}	Delete document from database and purge from Supabase storage.